Quarry: Security, Privacy and Compliance

Manoj Abhiram

November 10, 2026

CONTENTS

1. Why this matters 3

2. Assets and threat model 4

2.1 Assets 4

2.2 Actors 5

2.3 STRIDE summary 6

3. Authentication and sessions 7

4. Authorization 7

5. Outbound request safety (SSRF) 8

6. Secrets and cryptography 9

7. Application and transport hardening 9

8. Privacy by design 11

8.1 Data inventory 11

8.2 Controls that implement privacy rights 12

9. Legal and regulatory map 12

10. Incident response outline 15

11. Security verification 16

12. Known gaps and recommended fixes 16

Document	Threat model, security controls, privacy design and compliance posture
Version	1.0
Date	2026-11-10
Status	Controls described here are implemented. The legal analysis is **not legal advice**; it summarizes research notes and must be reviewed by counsel before a hosted launch

1. Why this matters

Quarry stores two kinds of sensitive data:

1. **The customer's secrets**: API keys, mailbox passwords, CRM tokens, payment state.
2. **Third-party personal data**: names, job titles, email addresses and profile links of people who never signed up. This is the larger obligation. Those people have rights (to be told, to object, to be erased) that Quarry's customers have to be able to honour through the product.

Security and compliance are therefore part of the product, not a launch checklist.

2. Assets and threat model

2.1 Assets

Asset	Sensitivity	Where it lives
User credentials and sessions	High	`users`, `user_sessions` (hashed)
Customer API and mailbox secrets	High	Encrypted fields in `users`, `outreach_mailboxes`, `provider_keys`, `crm_connections`
Lead and decision-maker data (third-party PII)	High	`leads`, `decision_makers`, `lead_enrichments`
Suppression ledger	High (legal effect)	`suppressions`
Billing state	High	`credit_ledger`, `subscriptions`, `billing_events`
Operator server keys (Serper, LLM, payments)	High	Environment only

2.2 Actors

Actor	Capability	Goal
Anonymous internet user	Hits public endpoints (login, opt-out, unsubscribe, webhooks)	Brute force, enumeration, abuse
Authenticated tenant	Valid session in one workspace	Read or change another tenant’s data
Hostile web page or search result	Content Quarry fetches and feeds to an LLM	SSRF to internal services; prompt injection
Hostile DNS server	Answers differently on repeated lookups	DNS rebinding past the address check
Malicious email recipient	Receives outreach	Forge an unsubscribe for someone else
Network attacker	Observes or replays traffic	Session theft, replayed payment webhooks
Compromised dependency	Runs in the API process	Exfiltrate secrets

2.3 STRIDE summary

Threat	Example	Mitigation	Residual
Spoofing	Forged identity header	No header identity shortcut exists; only a verified token or session is accepted	None known
Tampering	Client sets request status to Completed	`status` is server-controlled; client value ignored	None known
Repudiation	”I never erased that person”	Append-only `audit_log` of compliance actions	Audit log is in the same database
Information disclosure	Cross-tenant read; PII in logs; secrets in URLs	Ownership checks on every lookup; enriched payloads not logged at INFO; no tokens in URLs (SSE `?token=` exists only for clients that cannot send headers)	`?token=` can reach proxy logs; prefer the cookie
Denial of service	Many launches on one SQLite writer	Rate limits, plan and cost quotas, `num_leads` capped at 200 per request	Single-writer ceiling
Elevation	Viewer performs admin action	Per-endpoint minimum role	None known

3. Authentication and sessions

Control	Detail
Password storage	bcrypt with per-password salt
Access token	Short-lived JWT in an HttpOnly cookie (`quarry_access`); the token `type` claim must be `access`, so a refresh token is not accepted as an access token
Refresh token	JWT in an HttpOnly cookie scoped to `/api/auth`; only a hash is stored server-side; rotated on every refresh; revoked on logout
CSRF	Double-submit: cookie `quarry_csrf` (readable) must equal the `X-CSRF-Token` header on unsafe methods. Compared in constant time. Bearer clients are exempt because they hold no ambient credential
Cookie flags	`Secure` when the frontend is https (override `COOKIE_SECURE`), `SameSite=Lax` by default, `None` forces Secure, optional `COOKIE_DOMAIN`
Google sign-in	Server-side redirect flow; the callback sets cookies and redirects; no tokens in URLs
Brute force	Sliding-window rate limit on login, register, reset and verify; uniform error messages so login does not reveal whether an email exists
Password reset	Time-limited link sent by email (`PASSWORD_RESET_EXPIRE_MINUTES`), signed with a `password_reset` token type; the request endpoint answers identically whether or not the email exists. **Known gaps:** the token is a stateless JWT, so it can be reused until it expires, and a reset does not revoke existing sessions. See section 12
Startup	The server refuses to start without `SECRET_KEY`; there is no insecure fallback

4. Authorization

- **Object-level checks.** Every read or write of a request, lead, list, mailbox, sequence, webhook or key is scoped to the owner or to the active workspace. Adding a lead to a list checks both the list and the lead; list export filters by owner.

- **Roles.** `owner > admin > member > viewer`. A dependency factory (`teams/deps.py`) requires a minimum role per endpoint and returns 403 with the needed role named. Workspace is selected with `X-Org-Id` and verified against membership.
- **Sharing.** Sharing a request or list gives teammates read access through dedicated `/shared/` endpoints; it never widens write access.
- **Public API keys.** Stored only as a SHA-256 hash, shown once, prefixed `qk_`, revocable, and usable only on the read-only `/public/v1` routes.
- **Admin operations.** Billing top-ups, member management and integrations require admin.

5. Outbound request safety (SSRF)

Quarry fetches URLs it did not choose: search results, directory entries, company websites, LinkedIn public pages. Hosting it makes that a server-side request forgery risk. `core/url_guard.py` is the single guard.

Rule	How
Public addresses only	Refuse loopback, private, link-local and cloud-metadata ranges (for example `127.0.0.1`, `10.x`, `169.254.169.254`)
No DNS rebinding	Resolve the name once, check every returned address, then connect to the **checked IP** (httpx via a wrapped network backend, `requests` via per-hop resolution with the original Host header and TLS server name). TLS still verifies the real hostname
Redirect safety	Each redirect hop is re-resolved and re-checked
Coverage	Scraper, tech-stack detection, careers-page detection, LinkedIn fetchers and the MX check all use the guard. Fixed first-party endpoints (Serper, Google OAuth, outbound SMTP) are deliberately not guarded
Tests	`test_url_guard*.py` covers address classes, redirects to metadata IPs, MX hosts that resolve to private addresses, and fails loudly if an httpx or httpcore upgrade removes the private attribute the pinning relies on

Residual risks

- Playwright's browser does its own DNS: checked but not pinned. Playwright is off by default.
- The pinned HTTP client ignores environment proxy settings.
- There is no response-size cap in the scraper (address and redirect hops only).
- Pages are untrusted input to an LLM. Prompt injection from a scraped page could skew extraction. Mitigation: output is parsed as structured JSON, nothing the model returns is executed, and fields carry provenance so users can audit.

6. Secrets and cryptography

Secret	Protection
User Serper key, verifier key, provider keys (Hunter, Snov, PDL), mailbox SMTP and IMAP passwords, CRM tokens	Fernet (AES-128-CBC plus HMAC) with a key derived from `SECRET_KEY`; masked on read; validated with a live call when saved where possible
Public API keys	SHA-256 hash only
Invite and reset tokens	Stored hashed
Unsubscribe and tracking tokens	HMAC-signed, purpose-bound, so a token minted for one purpose is rejected for another
Outbound webhook signing secret	Stored encrypted; HMAC-SHA256 over `timestamp.body`
Job payloads	Never contain secrets; handlers load the key at run time

Limitation. The encryption key is derived from `SECRET_KEY`, so rotating `SECRET_KEY` makes stored credentials unreadable. Before hosting, move to a versioned data key held in a secret manager.

7. Application and transport hardening

- CORS is an explicit origin list; credentials are allowed only for those origins.
- Unhandled exceptions return a generic 500; details go to logs only.
- Scraped personal data is not logged at INFO.

- The API container runs as a non-root user (uid 10001) with a health check; the database volume is the only writable path.
- nginx proxies `/api` so cookies are same-origin; long timeouts and no buffering support server-sent events.
- TLS terminates in front of the containers. nginx here speaks plain HTTP, so **do not expose port 8080 directly to the internet**.
- Dependencies are pinned by `uv.lock` and `requirements.txt`; CI runs the full test suite on every push.

Recommended before hosting: HSTS and security headers (CSP, `X-Content-Type-Options`, `Referrer-Policy`) at the proxy; dependency vulnerability scanning in CI; secret scanning; backups of the data volume with a restore test.

8. Privacy by design

8.1 Data inventory

Data	Subject	Purpose	Source	Retention
Account details	The customer	Operate the service	The customer	Until account deletion
Lead company data	Organizations	Prospecting	Public web, public search results	Kept with the request; shared company facts may persist
Decision-maker name, title, email, profile URL	**Third parties**	Prospecting	Public web pages, public search results, paid providers the customer configured	Deleted with the request or account; rejected leads purged after 30 days
Outreach messages and replies	Third parties	Run sequences, detect replies and bounces	The customer’s mailbox	With the sequence
Suppression entries	Third parties	Honour opt-outs	Opt-out, unsubscribe, manual	Kept (needed to prevent re-contact)
Audit entries	Customer actions	Accountability	System	Kept; holds no raw personal values

8.2 Controls that implement privacy rights

Right or duty	Control
Transparency / source notice	Per-field provenance (source URL and collection date) in the UI; exports carry a source-notice line and opt-out instruction; a public `/privacy` page
Objection / opt-out	Public opt-out page and signed links; one-click unsubscribe headers; opt-outs write to the suppression ledger
Suppression is enforced	The ledger is checked **before enrichment, export and send**, by email and by whole domain
Erasure	`/compliance/erase` removes a person across the workspace; account deletion cascades; an audit entry records the action without the raw value
Storage limitation	Rejecting a lead starts a 30-day clock; `compliance.retention` purges rejected leads and un-promoted search results (and the people in them) after 30 days, runnable on a schedule or on demand with `--dry-run`
Regional handling	Region flag per lead (EU, UK, CA, IN) with a note, inferred from country, location text or domain suffix
Accountability	Append-only audit log, visible in the Compliance page
Data minimization	Search results are not promoted to leads until reviewed; sensitive provider keys never leave the server

9. Legal and regulatory map

> Summary of research notes, mostly law-firm commentary and not primary texts. It is a checklist for a lawyer, not a conclusion.

Regime	Relevance to Quarry	What the product does	Open question
GDPR (EU)	Lead subjects in the EU; CJEU (Oct 2024) accepts purely commercial interests as a possible legitimate interest, subject to the three-part test	Source notice, opt-out, erasure, region flag, suppression	Legitimate-interest assessment, Art. 14 notice duty for the operator, DPA and subprocessor list
UK GDPR / PECR	Sole traders are treated as individuals for electronic marketing	Region flag UK; suppression	Subscriber-type handling (company vs sole trader)
CAN-SPAM (US)	No B2B exemption; needs a valid postal address and a working opt-out; per-email penalties	Footer with postal address and unsubscribe; one-click headers; suppression honoured at send	Customer must set `OUTREACH_POSTAL_ADDRESS`; terms must put sending duties on the customer
CASL (Canada)	Requires documented implied or express consent; sender bears the burden of proof	Region flag CA with a note	No consent record is captured
CCPA / CPRA and state data-broker laws	B2B exemption expired 1 Jan 2023; California brokers handle deletions via the DROP system;	Per-customer processing by default; deletion tooling	**Whether a shared, retained database makes Quarry a data broker.** The `companies` table is shared across users but holds company facts, not people

Regime	Relevance to Quarry	What the product does	Open question
	first enforcement action announced 13 Aug 2026		
India DPDP Act and Rules	Notified Nov 2025; obligations phase in toward about May 2027; no general legitimate-interest ground	Region flag IN with a note	Scope for publicly available B2B contact data is unresolved
LinkedIn terms	Platform litigation ended Proxycurl (2025); hiQ lost on contract and fake-account facts	Public search results only, no logged-in scraping	Terms risk for any LinkedIn-derived field
Email provider rules	Google and Microsoft sender requirements; complaint rate targets (0.10% goal, 0.3% limit)	Caps, warmup, auto-pause, SPF/DKIM/DMARC preflight, unsubscribe headers	Per-user volumes are far below bulk-sender thresholds, but the protections help the customer's domain

Decisions recorded

- Per-customer processing is the default posture. A shared, resold contact database is a non-goal.
- Mailboxes connect by SMTP and IMAP, not OAuth, to avoid restricted Gmail scopes.
- LinkedIn is used only through public search results.

Required before a hosted launch

1. Counsel review of data-broker status, legitimate-interest assessment, DPA, subprocessor list and terms of service.
2. Publish the final privacy notice and terms; set `OUTREACH_POSTAL_ADDRESS` for every sending customer.
3. Decide and document the retention schedule the operator will run.
4. Define a breach-response procedure and a contact for data-subject requests.

10. Incident response outline

Step	Action
Detect	Alerts on 5xx rate, job failure rate, auth failure spikes (to be added)
Contain	Rotate `SECRET_KEY` (invalidates sessions; credentials must be re-entered), revoke provider keys, disable the affected mailbox or webhook
Assess	Use `audit_log` and application logs; determine which tenants and which third-party records were affected
Notify	Customers affected, and regulators where required (for example within 72 hours under GDPR when personal data is breached)
Recover and learn	Restore from backup, patch, add a regression test

11. Security verification

Area	Test evidence
Cookie auth and CSRF	`test_auth_cookies.py`
Ownership and flow hardening	`test_flow_hardening.py`, `tests/db/test_hardening.py`, `test_dependencies.py`
SSRF guard and pinning	`test_url_guard*.py` (six modules)
Compliance	`tests/phase2/test_compliance.py`, `test_provenance_region.py`
Billing idempotency and webhook signatures	`tests/growth/test_billing.py`, `test_topup.py`
Outreach footer, unsubscribe and tokens	`tests/p3/test_outreach_footer.py`, `test_p3_outreach_logic.py`
Deployment env	`test_deployment_env.py`

Not yet done: an external penetration test, dependency vulnerability scanning in CI, and a review of the live-service paths (payments, OAuth, real SMTP) against real providers.

12. Known gaps and recommended fixes

These were found while writing this document by reading the code. None is a regression; each is a hardening step.

#	Gap	Risk	Recommended fix	Priority
1	Password-reset token is a stateless JWT: reusable until expiry	A leaked reset email link works more than once	Store a one-time token hash (or a password-version claim) and invalidate on use	P2
2	Password reset does not revoke existing sessions	An attacker holding a stolen session survives a password change	Delete all `user_sessions` for the user on reset and on password change	P2
3	No cap on concurrent pipeline runs per user	One user can queue many runs against one SQLite writer	Limit active runs per user; the plan and cost quotas already bound monthly volume	P3
4	No response-size cap in the scraper	A hostile page could return a very large body	Stream with a byte limit	P3
5	Encryption key derived from `SECRET_KEY`	Rotation orphans stored credentials	Versioned data key from a secret manager	P2 before hosting
6	SSE accepts `?token=`	Token can appear in proxy logs	Prefer cookie auth; strip the query string from access logs	P3
7	No security headers at the proxy; no dependency or secret scanning in CI	Missing defence in depth	Add CSP, HSTS, `Referrer-Policy`; add `pip-audit`, `npm audit`, secret scanning	P2 before hosting